# LABORATORY 3

## Discrete-Time Control System Design using SIMULINK

轴流风扇的转速是所施加电压的线性函数，而精确的通风速率传感器则确定气流速率。

One reason for the construction of the environmental test chamber in the Engineering Department at Lancaster University, is to investigate the utility of new control systems for the regulation of ventilation rate (Tate *et al*. 2018; Tsitsimpelis & Taylor 2014, 2015). Here, the rotational speed of an axial fan is a linear function of the applied voltage, while an accurate ventilation rate sensor determines the airflow rate. The whole system is monitored by a PC with either LABVIEW and/or MATLAB software for on-line control.

The discussion in Laboratories 1 and 2 centred on the ventilation rate model without any control. Now consider an automatic feedback controller as illustrated in the block diagram Figure 1. Here, $y_d(k)$ represents the desired ventilation rate, i.e. the airflow that we want in the room. This variable is also called the command input, reference level or set point.

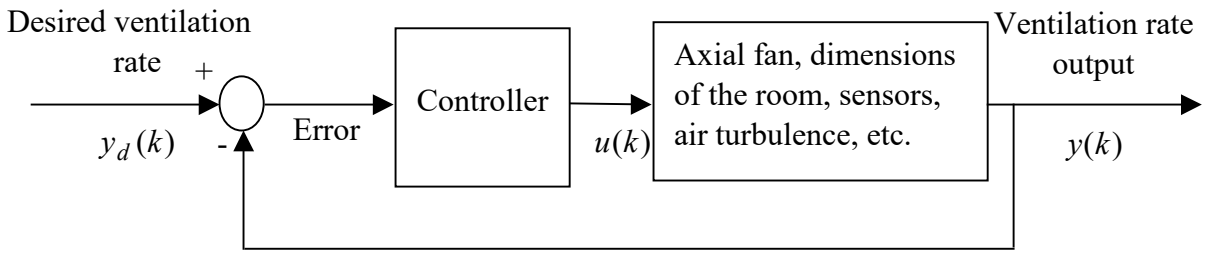


**Figure 1:** Automatic feedback controller for ventilation rate.

The actual ventilation rate $y(k)$ is subtracted from the desired level $y_d(k)$, to form the error signal given to the controller. The objective is to minimise this error as much as possible. For example, we might specify a ventilation rate of $y_d(k) = 300$ m$^3$/hour. In this case, the automatic controller must adjust the fan voltage to maintain $y(k) = 300$ m$^3$/hour.

The controller is a mathematical algorithm programmed into the computer: it takes the error signal and calculates the required voltage. Note that it doesn't necessarily have to be a very complex algorithm! For example, in the case of *proportional control* shown in Figure 2, it is simply a scalar gain, i.e. a number $k_p$ chosen by the engineer.

For the purposes of the simulation-based exercises, we replace the real ventilation rate system with a mathematical model: a first order discrete-time transfer function with unity time delay, as in Figure 2.

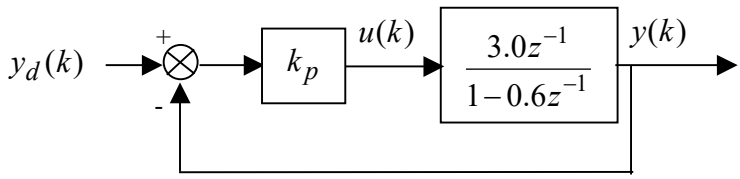


**Figure 2:** Block diagram showing proportional control.

The problem is: how do we choose a suitable value for the control gain $k_p$?

In the simplest case, we will use one of the most common methods prevalent throughout industry, namely trial and error. Here, the control gain is adjusted on-line until everything seems to work, i.e. the control gains are 'tuned by hand'.

**Note on the time delay and units.** The ventilation model utilised in Laboratory 1 has two samples time delay, i.e. $\delta = 2$. However, for the purposes of the present workshop, we will initially

assume a delay of 1 sample, i.e. $\delta = 1$, as shown above. Also, these notes assume the output is measured using units of m$^3$/hour while the input is the applied voltage to the fan expressed as a percentage, i.e. between zero and 100%.

## Exercise 3.1 Proportional control of ventilation rate

The present exercise assumes a basic familiarity with SIMULINK. Note also that SIMULINK operates quite differently from version to version. Partly for this reason, there is no attempt to describe each step of the exercise in these notes.

Create the block diagram shown in Figure 2. Using SIMULINK your block diagram might look like that shown in Figure 3. For the purpose of this module, you should use the **Discrete Filter blocks** based on the backward shift operator i.e. not the Discrete Transfer Function block, which is based on the forward shift operator.

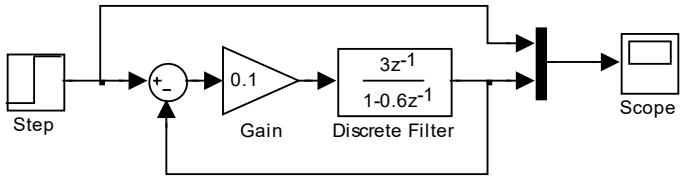


**Figure 3:** SIMULINK diagram of Proportional control.

Double click on the *Gain* block and enter a value of 0.1 as shown in Figure 3. This is a reasonable initial value to try for the proportional control gain $k_p$.

Also, enter the parameters for the system transfer function, i.e. double click on the Discrete Filter and enter $[0, 3]$ for the numerator and $[1, -0.6]$ for the denominator.

Double click on the *Sum* block and change the dialogue box from |++ to |+- (i.e. vertical bar, plus, minus) so that your diagram employs negative feedback as required.

Double click on the *Step* block and change the Final Value from 1 to 300. This represents the set point for this exercise, i.e. $y_d(k) = 300$ m$^3$/hour. At this point, it is worth recalling that in Laboratory 1 the **step input represented the voltage to the fan**, expressed as a percentage. Here, however, the step input is something entirely different, it is **the desired ventilation rate**, with units of m$^3$/hour.

Run the simulation and examine the results using the *Scope*. Hopefully, your plot will show the set point jump from an initial 0 to 300 m$^3$/hour, while the ventilation rate will settle out some way below 300. This is the problem with proportional control: there is always a steady state error between the actual output and the desired level.

Try using $k_p = 0.05$ and $k_p = 2$ by changing the setting in the *Gain* block.

What is the effect on the steady state error and the speed of response? What happens if you keep increasing the value of $k_p$ to reduce the steady state error?

Whilst completing the above, add another *Scope* to the block diagram so that you can also examine a plot of the control input. This *Scope* would pick off the signal between the *Gain* and the *Discrete Filter*. The control systems designer should always look at the control input signal, even though the necessary *Scope* is not actually shown on this handout – *mea culpa*. Keep this in mind during any later exercises.

## Exercise 3.2 Integral control of ventilation rate

Integral control involves integrating the error signal as well as multiplying it by a gain. This ensures that (as long as the closed loop is stable) the output *always* eventually reaches the desired level, or more formally stated: the output asymptotically approaches the desired level.

**Step 1.** Modify the block diagram from above to represent integral control, as shown in Figure 4.

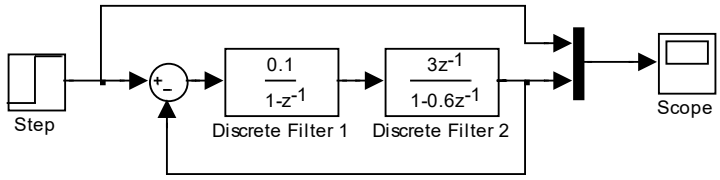


**Figure 4:** SIMULINK diagram of Integral control.

Note that the control algorithm is another *Discrete Filter* block.

Enter a value of 0.1 for the integral gain as shown above. The denominator of this Discrete Filter should be $1 - z^{-1}$, represented in the dialogue box by $[1, -1]$.

Ensure that the final value of the step is 300 as before. Run the simulation and examine the results using the *Scope* block.

If the output hasn't settled down to a constant (steady state) value by the end of the simulation, then you need to change the simulation length. From the 'Simulation' menu of your model, select 'Parameters…' to open a dialogue box. Change 'Stop time' to say 30.

Investigate different values of $k_I$, checking that the steady state error is always zero. As before, you should find that a higher value of $k_I$ yields a faster response. Unfortunately, sufficiently large values of $k_I$ also lead to oscillations in the output which we generally do not want.

Find the maximum value of $k_I$ (approximately) that does not yield an overshoot. How does this response compare with the equivalent proportional control case? When looking at the speed of response, remember to check the *x*-axis on the Scopes in case you are using a different simulation length from the previous examples.

Better still, save the data to the workspace using the Scope block, and plot the results using a MATLAB script. In this way, you can add a legend and ensure a professionally presented plot.

## Exercise 3.3 Proportional-Integral control of ventilation rate

Proportional-Integral (PI) control combines the single control actions of the previous two exercises. For many first order systems, PI control offers a fast non oscillatory response with zero steady state error. The preliminary example shown below is used to introduce the PI controller. It is based on the same model as above and uses an illustrative set of control gains.

Modify your block diagram from above to represent a PI control system with $k_I = 0.14$ and $k_p = 0.16$, as shown in Figure 5.

Compare the response with the examples above. Try different values of $k_I$ and $k_p$.

Rather than conduct potentially expensive experiments on the real system, a model is often used to tune the control gains before practical implementation. In the case of the ventilation system above, a good, fast and stable response in SIMULINK is obtained using e.g. $k_I = 0.14$ and $k_p = 0.16$.

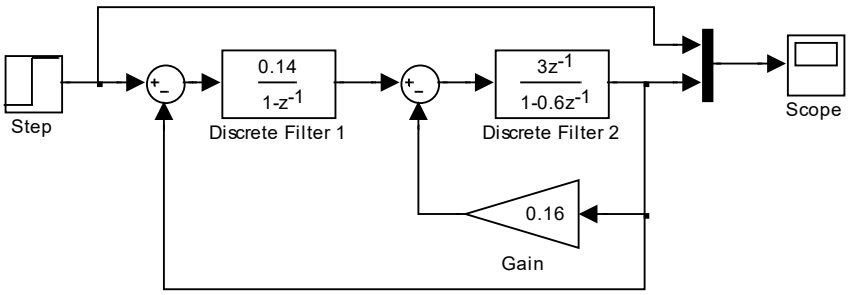


**Figure 5:** SIMULINK diagram of Proportional–Integral (PI) control.

Of course, this approach assumes that our mathematical model is an accurate representation of the real system. By contrast, Exercise 3.4 simulates what happens if the model has been incorrectly estimated.

## Exercise 3.4 Control of ventilation rate with model mismatch

The control gains $k_I = 0.14$ and $k_p = 0.16$ were estimated using the following model:

$$y(k) = \frac{3z^{-1}}{1-0.6z^{-1}}u(k) \tag{1}$$

using a pole assignment design with closed loop poles of 0.3 and 0.4 on the unit circle.

Use the SIMULINK model from Exercise 3.3 above. However, now assume that the denominator of the system is actually $1-0.9z^{-1}$, i.e. enter [1, –0.9] in the dialogue box for the system, leaving the controller unchanged.

Simulate the block diagram and examine the response as normal.

You should find that the ventilation rate overshoots the set point slightly. Note, however, that the output nonetheless reaches the required 300 m$^3$/hour quite quickly – this is always the case for controllers that include integral action. In fact, getting the mathematical model a little wrong doesn't have much effect in this particular example: *this is the advantage of using feedback*.

Now try a more serious example of model mismatch by increasing the time delay of the system. Restore the system denominator to $1-0.6z^{-1}$, but change the numerator to represent two samples time delay, i.e. $3z^{-2}$ or [0, 0, 3].

The oscillatory response observed is typical when a PI controller is applied to a system with longer than unity time delays and is one reason for using higher order control systems, such as the Proportional-Integral-Plus (PIP) approach developed in the lectures (Taylor *et al*. 2013).

## Exercise 3.5 Pole Assignment Design

This section is based on the pole assignment approach to control system design, as introduced in the set of lecture slides called "4. Discrete–Time Control & Pole Assignment". Although based on an axial fan, it must be a different fan to that considered in the above exercises (or perhaps the same fan but calibrated differently) hence the parameter values are different!

In SIMULINK, investigate the response of PI control using the axial fan control model quoted in the lectures, i.e. with model coefficients as follows: $a_1 = -0.8$ and $b_1 = 1.6$.

A deadbeat control system based on placing the closed-loop poles at the origin of the unit circle is obtained using a proportional gain $f_0 = 0.5$ and an integral gain $k_I = 0.625$.

Another illustrative pole assignment design considered in the lectures was based on a complex conjugate pair of poles, namely $0.6 \pm 0.3j$, yielding $f_0 = 0.2188$ and $k_I = 0.1563$.

By entering appropriate values for the control gains and model coefficients, and using a similar SIMULINK diagram as for Exercise 3.3, reproduce the figures shown in the lecture slides i.e. the slides that introduce the pole assignment method. These are also reproduced below in Figure 6: the deadbeat response is shown on the left and the alternative design on the right.

Examine the input and output signals from your own *Scopes* to check that everything is working correctly i.e. to obtain the same response as on the slides.

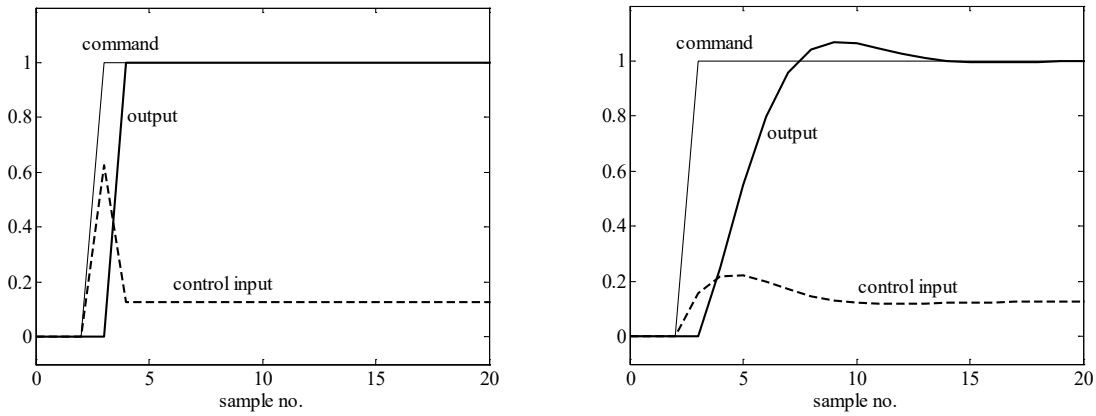


**Figure 6:** Time response based on deadbeat (left) and a complex conjugate pair of poles.

Next, add a step disturbance of magnitude 1.0 to $y(k)$ after 20 samples and compare the response of the two control systems from above. Since the disturbance will be programmed to start after 20 samples, you will need to increase the simulation length to e.g. 40 samples. The new step block (representing a disturbance) should be connected to a new summation block placed immediately after the system Transfer Function. The control feedback signals should be picked off after this disturbance. In other words, the controller will be using the measured output that is influenced by the disturbance. The disturbance represents an unmeasured input to the system e.g. a change in the external wind speed or, more likely for this laboratory example, a door being opened.

Finally, consider the relative robustness of the two control systems from Step 1, by redefining $a_1 = -0.7$ while keeping the control gains the same. This represents a severe mismatch between the estimated model (used for control system design) and the system actually being controlled. You might leave the above step disturbance in place for this example i.e. to simultaneously investigate disturbances and model mismatch. Investigate other model mismatch examples e.g. try using $b_1 = 2.0$ and $a_1 = -0.7$ as the simulated model (model mismatch in both the numerator and denominator).

Armed with the responses from steps 2 and 3 above, you should be able to answer the following question. *What are the disadvantages of deadbeat design?*

## References

Tate, O., Wilson, E.D., Cheneler, D. and Taylor, C.J. (2018) Computational fluid dynamics and data–based mechanistic modelling of a forced ventilation chamber, *18th IFAC Symposium on System Identification* (SYSID), 9–11 July, Stockholm, Sweden. Publisher: Elsevier, *IFAC–PapersOnLine*, **51**, 15, pp. 263–268.

Taylor, C.J., Young, P.C. and Chotai, A. (2013) *True Digital Control: Statistical Modelling and Non–Minimal State Space Design*, John Wiley & Sons Ltd.

Tsitsimpelis, I. and Taylor, C.J. (2014) A 2–dimensional Hammerstein model for heating and ventilation control of conceptual thermal zones, *10th UKACC International Conference on Control*, July, Loughborough, UK. Publisher: *IEEE.*

Tsitsimpelis, I. and Taylor, C.J. (2015) Partitioning of indoor airspace for multi–zone thermal modelling using hierarchical cluster analysis, *14th European Control Conference*, 15–17 July, Linz, Austria. Publisher: *IEEE.*